Komikan Concurrente

Informe técnico de sincronización · revisión de auditoría

1. Arquitectura y recursos compartidos

Un proceso simula N clientes locales mediante hilos. Cada cliente representa un bando; no es una pieza. Un emparejador consume una cola FIFO de llegadas y reserva hasta M mesas. Cada mesa activa tiene un juez independiente y trece hilos de piezas, con retardos locales aleatorios entre 50 y 300 ms. Los clientes y el emparejador esperan mediante variables de condición, sin espera activa.

2. Exclusión mutua y validación del juez

El mutex del torneo protege la cola, asignaciones, contadores y estados de slots. Cada mesa tiene otro mutex que protege su tablero, estado terminal y buzones de intenciones. Las piezas calculan un destino desde una vista consistente y publican una intención con secuencia. El juez consume las solicitudes en ese orden, revalida adyacencia, ocupación y captura contra el estado vigente y aplica la jugada en una única sección crítica. Si dos intenciones reclaman una casilla, solo la primera todavía válida puede ocuparla.

3. Reglas e invariantes

El tablero es un grafo explícito de 28 nodos: 25 de cuadrícula y tres del trono. Se siguen las líneas y círculos de la figura; no se añade una posición en el cruce sin círculo ni diagonales en la cuadrícula. Un salto recorre dos aristas colineales hacia adelante. Se acepta longitud gráfica desigual en la columna central del trono, una ambigüedad documentada. El tablero conserva una posición por pieza viva, una pieza por nodo y perros vivos + capturados = 12. El Puma gana con seis capturas; los Perros ganan si no hay paso ni salto legal.

4. Finalización y propiedad de los hilos

El servidor crea una vez las primitivas de cada slot antes de exponerlo a clientes y señales. El juez inicializa el tablero de cada partida, habilita su estado board_ready y crea las piezas; no se reinicializan mutexes que podrían recibir notificaciones tardías. El juez une sus piezas antes de avisar al torneo; el emparejador une al juez antes de reutilizar el slot. SIGINT se bloquea como primera operación de main y sigwait se inicia antes de configuración y reservas. Una bandera atómica conserva interrupciones de inicio; bind/unbind sincroniza el contexto del callback. El cierre despierta esperas y une hilos antes de destruir recursos. SIGUSR1 despierta al hilo de señales al terminar normalmente.

Riesgos y evaluación

5. Deadlock, livelock y starvation

Los mutexes de torneo, tablero y salida no se mantienen anidados. El registro de eventos se hace tras copiar el tablero y liberar su mutex. Las esperas temporizadas usan CLOCK_MONOTONIC y liberan el mutex; los despertares espurios preservan el plazo original. Todos los predicados se revisan en bucles. El juez no espera un turno ni a una pieza concreta: consume cualquier intención pendiente. Esto evita ciclos de espera y una ficha inmóvil no detiene al resto.

Los reintentos se espacian con retardos aleatorios; no hay bucles de sondeo continuo. La cola FIFO evita adelantar clientes ya admitidos y las intenciones se atienden por secuencia. POSIX no promete equidad estricta al adquirir mutexes ni planificación justa: no se puede demostrar ausencia absoluta de starvation. Tampoco se garantiza que el movimiento aleatorio alcance una victoria en tiempo finito. El PDF no define empate ni límite temporal; SIGINT permite cerrar esas ejecuciones sin inventar ganadores.

6. Fallos y abandono

Se distinguen abandono, error de infraestructura y salida inesperada de participante. Cleanup handlers conocen si el propio hilo posee el mutex, retiran su intención y notifican al juez, que declara ganador al rival. Se prueban pthread_exit con y sin mutex y cancelación diferida de piezas en cond_wait, que recupera el mutex antes del cleanup. La cancelación se deshabilita durante transiciones de propiedad, registro e infraestructura; nunca se desbloquea un mutex ajeno. Los errores de creación provocan cierre con join parcial. No se recuperan cancelación asíncrona, SIGSEGV, SIGKILL ni corrupción arbitraria del proceso.

7. Complejidad, rendimiento y concurrencia

Se usan N hilos de clientes, hasta 14 por mesa activa y tres contextos de control. Se reservan min(M, N/2) slots: memoria O(N + min(M, N)). Se retiraron los máximos arbitrarios de 1024 clientes y 64 mesas; se validan size_t, productos de reserva y plazos, y se atienden ENOMEM/EAGAIN. Esto no promete capacidad ilimitada. Un mutex por tablero simplifica la atomicidad; el juez serializa modificaciones locales, mientras piezas y mesas progresan concurrentemente. El log completo facilita auditoría y puede limitar rendimiento. No se presentan cifras de throughput o latencia no medidas.

8. Evidencia y límites de verificación

La auditoría ejecuta GCC C17 con -Wall -Wextra y -pthread, unitarios y 22 grupos de integración: 2, 10, 100 y 200 clientes, reciclaje, FIFO, límite M, fallos y SIGINT mediante barreras de inicialización. La topología se contrasta en los 784 pares de nodos y 70 capturas dirigidas; una secuencia de seis capturas verifica victoria real. ASan/UBSan y TSan se aplican a las suites compatibles; comandos, resultados y Actions están en docs/pruebas.md. LSan local está limitado por ptrace; CI lo ejecuta fuera de ese trazador. No se extrapola ausencia de fugas a todos los interleavings. Antes del bloqueo inicial de señales y ante E/S no interrumpible del sistema no se promete cierre controlado acotado.